Go Clean Architecture for Beginners

Charlie Chiu 2025-8-9

前情提要

背景

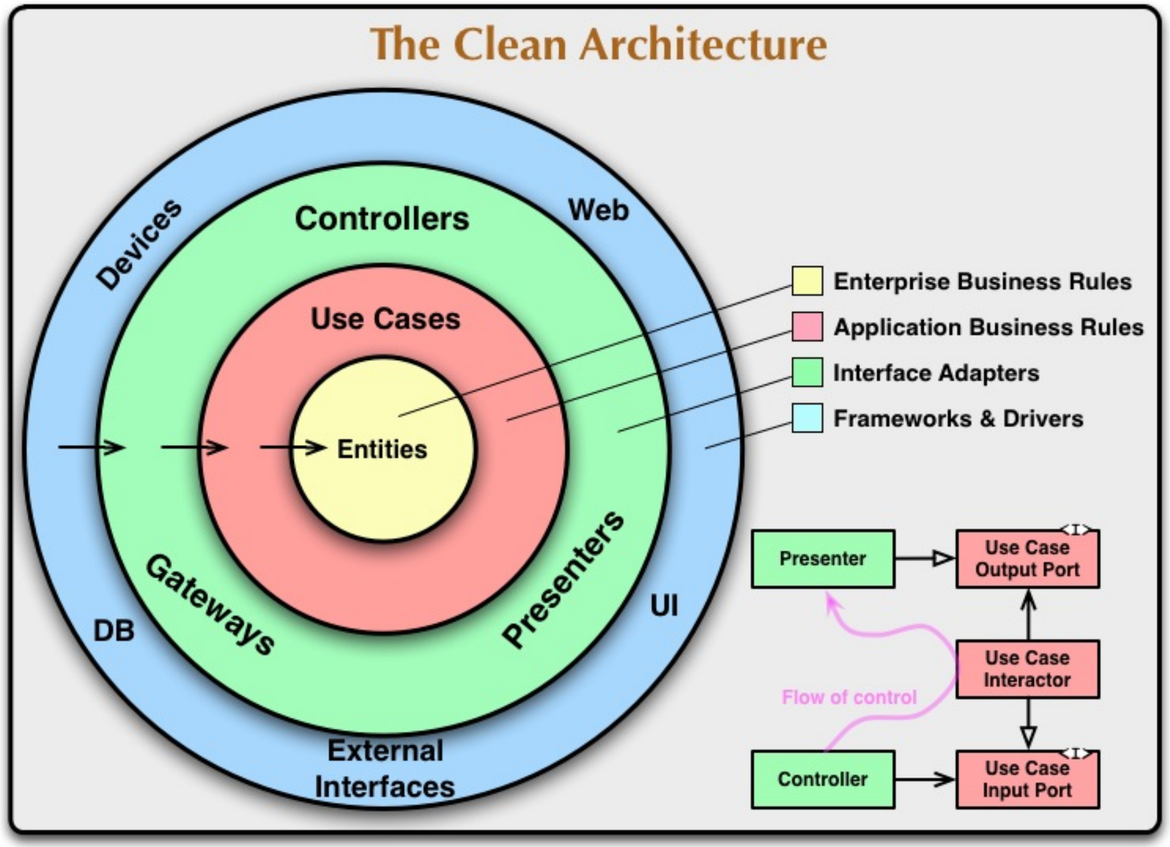
目標

- 可以交付「統一框架」的任務
- 框架的使用者是來自五個專業背景及 Golang 經驗各不相同的團隊，大約由50個開發者組成
- 用於重構現有產品並長期維護，穩定性重於快速迭代
- 有明確的設計理念，讓使用者可以很更容易的理解

期望的特性

- 有明確的指引
- 沒有強烈的個人風格
- 盡可能依循社群及開源風格
- 盡可能使用內建的 library 來避免「哪個比較好」的爭論
- 留下空間讓不同的團隊依需求發揮

Clean Architecture



Clean Architecture

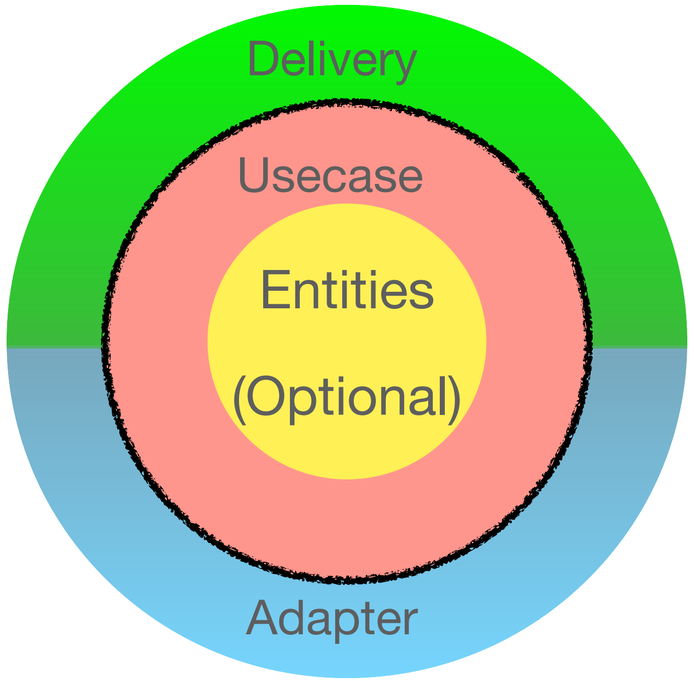
Input Requests

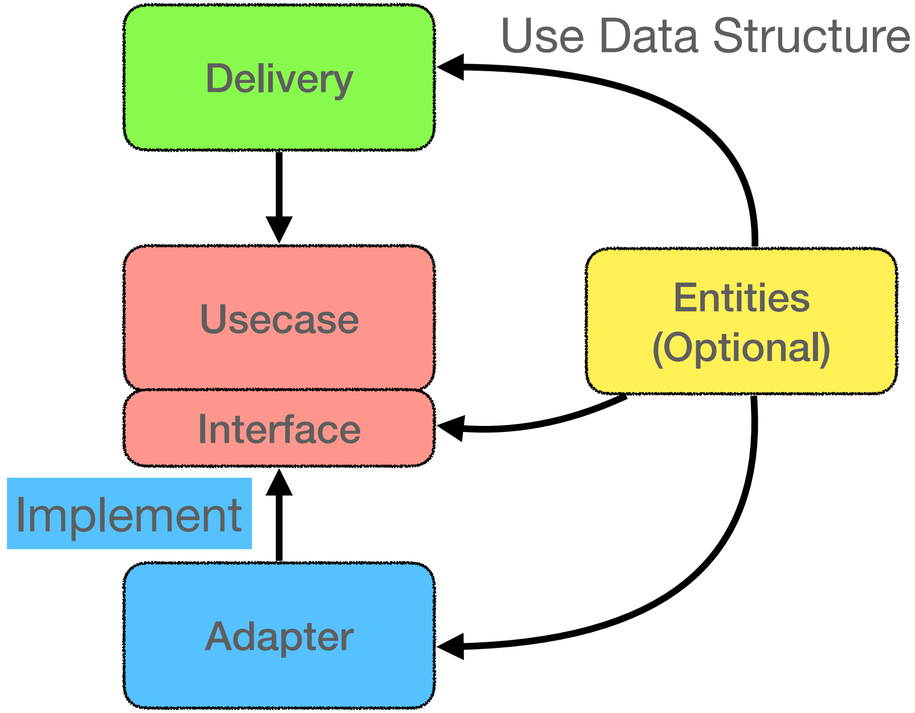
HTTP, cli

Business Logic

Output Requests

RDBMS, Cache, APIs, etc





- 「維持正確的依賴關係」👉「實作細節不可越界」

實作

usecase.Application & delivery layer

usecase

```go
/.../
type Application struct {  28 usages  new *
    logger *slog.Logger

    AuthService AuthService

    blogRepo BlogRepository // interface
}

// CreateArticle creates a new blog article in draft state
func (a *Application) CreateArticle(  6 usages  Charlie Chiu *
    ctx context.Context,
    title, content, authorID string,
) (
    *entity.Article,
    error,
) {...}
```

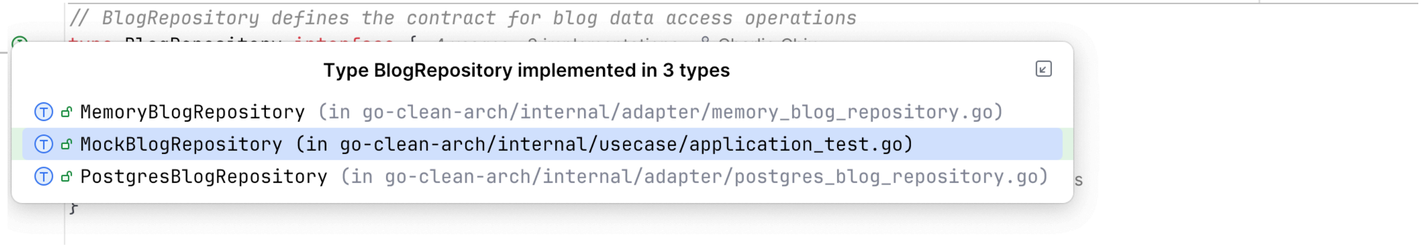
delivery.http

```go
// CreateArticle returns a handler for creating a new article
func CreateArticle(app *usecase.Application) gin.HandlerFunc {  2 usages  Charlie Chiu
    return func(c *gin.Context) {
        var req CreateArticleRequest
        if err := c.ShouldBindJSON(&req); err != nil {
            c.JSON(http.StatusBadRequest, gin.H{"error": "Invalid request body", "details":
            return
        }

        article, err := app.CreateArticle(c.Request.Context(), req.Title, req.Content, req.A
        if err != nil {
            c.JSON(http.StatusInternalServerError, gin.H{"error": "Failed to create article"
            return
        }

        c.JSON(http.StatusCreated, gin.H{"article": article})
    }
}
```

實作

usecase.Application & Adapter Layer

```go
// BlogRepository defines the contract for blog data access operations
type BlogRepository interface {  4 usages  3 implementations  Charlie Chiu
    Create(ctx context.Context, article *entity.Article) error  3 implementations
    GetByID(ctx context.Context, id string) (*entity.Article, error)  3 implementations
    Update(ctx context.Context, article *entity.Article) error  3 implementations
    Delete(ctx context.Context, id string) error  3 implementations
    List(ctx context.Context, filter BlogFilter) ([]*entity.Article, error)  3 implementations
}
```




```go
// AuthService defines the contract for authentication operations
type AuthService interface {  3 usages  2 implementations  Charlie Chiu
    LoginWithPassword(username, password string) (*AuthResult, error)  2 implementations
    RegisterWithPassword(username, password string) (*AuthResult, error)  2 implementations
    ValidateToken(token string) (*entity.User, error)  2 implementations
}
```

實作 - Layout

```
./
├── CLAUDE.md
├── cmd
│   ├── admin
│   │   └── main.go
│   └── api
│       └── main.go // 讀取 config 、初始化、 DI 把所有底層粘在一起
├── internal
│   ├── adapter
│   ├── config
│   ├── delivery
│   ├── entity
│   ├── platform
│   └── usecase
├── README.md
└── SPEC.md
```

https://github.com/golang-standards/project-layout

Recap

- 背景：支援不同程度的開發者協作，維護重於快速開發的框架
- 以 Clean Architecture 為出發點，規則精煉為「遵循正確的依賴方向」
- 用看起來一樣的 Layout 交付「統一框架」，並留下足夠的空間讓團隊自由發揮
- 不預設實作細節，大部份的組件都可以輕鬆的替換
- 提供足夠的設計資訊，讓 Google 或 AI Coding 更容易

Thank You

GitHub repo